AGILE PROJECT REPORT

University Management System

UMS
University Management System

Product Backlog, Prioritization & Sprint Planning

CSE342 — Agile Software Development

Youssef Tarek	23p0177
Marwan Ahmed	23p0242
Youssef Haddad	23p0113
Ziyad Elnemr	23p0013
Loay Khaled	23p0340
Ahmed Sadik	23p0140

Project Links: Jira: `https://ziyadelnemr90.atlassian.net/jira/software/c/projects/FMAP/boards/35` GitHub: `https://github.com/Ziyad-Elnemr/luemy`

Scrum Roles: Product Owner: Loay Khaled | Scrum Master: Youssef Tarek | Development Team: Youssef Haddad, Marwan Ahmed, Ziyad Elnemr, Ahmed Sadik

Academic Year 2025–2026

Presented TO:

Prof Mohamed Hassan Mahmoud El Gazzar

Eng Abdelrahman Salah

Eng Hazem Zaki

≡ Table of Contents

Project Overview

About This Document

This document covers Sprint 2 of the FMAP University Management System: prioritization (MoSCoW), the reconciled Product Backlog, MVP Components from Sprint 1 and Sprint 2, Sprint 2 meeting minutes, user story splitting techniques, estimation technique (Planning Poker), Definition of Done (DoD) adapted to the repository, and verification evidence.

System Description

The University Management System (UMS) is a digital platform to support students, professors, staff and administrators. Modules: Facilities, Curriculum, Staff, Community.

Agile Principles, Values & Pillars

The team follows Agile principles and values and intentionally applies the three Scrum pillars to guide delivery and process improvements. Evidence-based links to project artifacts are provided to show how principles are applied in practice.

Agile Pillars

- **Transparency**: Project work, board state and documentation are publicly available in the repository and docs folder (see GitHub and `docs/WORKFLOW.md`).
- **Inspection**: Regular inspection is implemented via daily scrums, sprint reviews and API documentation used for verification (see staff API docs: apps/staff/pages/api-docs.vue).
- **Adaptation**: The team adapts scope and process based on feedback; developer-facing process guidance lives in `docs/WORKFLOW.md` which documents how we work and how changes are handled.

How we demonstrate principles (evidence)

- Adaptability: see [Developer Workflow](docs/WORKFLOW.md) for developer-facing processes and change handling.
- Inspection: staff-facing API documentation is implemented in the app (apps/staff/pages/api-docs.vue) and rendered markdown (API docs page) for reviewers and QA.
- Transparency: source code, PRs and issue tracking available on GitHub: `https://github.com/Ziyad-Elnemr/luemy` and project Jira board.

Scrum Team Roles

Role	Assigned Member(s) and Responsibilities
Product Owner	Loay Khaled — defines and prioritizes the product backlog, represents stakeholders, accepts completed work.
Scrum Master	Youssef Tarek — facilitates ceremonies, removes impediments, ensures the team follows the Definition of Done.
Development Team	Youssef Haddad, Marwan Ahmed, Ziyad Elnemr, Ahmed Sadik — cross-functional engineers responsible for implementation, testing, and documentation.

Feature Prioritization

Prioritization Technique: MoSCoW Method

The team adopted the **MoSCoW prioritization method** — a widely-used Agile technique that classifies all Product Backlog Items into four categories based on business value, technical dependencies, stakeholder impact, and delivery risk.

M — Must Have

Non-negotiable features that the system cannot go live without. These represent core functionality essential to all primary user journeys. If any Must Have is missing, the release is considered a failure.

S — Should Have

High-value features that are important but not critical for the initial release. The system will function without them, but their absence significantly impacts user experience. These are high candidates for the next sprint.

C — Could Have

Nice-to-have features that improve usability or provide added convenience. Included only if time and resources allow within the sprint budget. Minimal impact if omitted.

W — Won't Have (this time)

Out-of-scope features for the current release cycle. They are acknowledged and documented for future sprints. This category also covers high-complexity items where cost/benefit does not justify immediate inclusion.

Product Backlog (reference)

All detailed backlog items are managed in the team's Jira board and linked from GitHub issue/PRs. The full backlog table has been intentionally omitted from this report to avoid duplication and ensure the single source of truth remains the Jira board.

Backlog Reference

Jira board: `https://ziyadelnemr90.atlassian.net/jira/software/c/projects/FMAP/boards/35`
GitHub repository: `https://github.com/Ziyad-Elnemr/luemy`
The backlog and live prioritization are maintained on the Jira board; for verification see linked PRs and issue references on GitHub.

MVP Components — Sprint 1 & Sprint 2

Sprint 1 Items

Sprint 1 Deliverables

- FMAP-133 — Create a New Course
- FMAP-135 — Enroll in a Selected Course
- FMAP-137 — View the List of Active Courses
- FMAP-138 — Create a New Assignment with Deadlines
- FMAP-141 — Upload and Submit Assignment Documents
- FMAP-147 — Grade an Assignment and Provide Feedback
- FMAP-160 — Registration & Login for Student

Sprint 2 Items

Sprint 2 Deliverables

- FMAP-131 — Update Existing Course Details
- FMAP-132 — Search and Filter the Course Catalog
- FMAP-140 — View Upcoming Assignment Deadlines
- FMAP-178 — Manage Availability for Professor
- FMAP-184 — View Centralized Staff Directory
- FMAP-198 — Update Student Personal Information

Sprint	Completed Items
Sprint 1	7
Sprint 2	6
Total Completed	**13**

Repository Folder Structure

Project Organization

The FMAP (University Management System) repository is organized as a monorepo with clear separation of concerns:

```
luemy/ (root)
 apps/                      # Multi-app frontend structure
    students/             # Student application (Nuxt 3)
    staff/                # Staff/Admin application (Nuxt 3)
    instructors/          # Instructor application (Nuxt 3)
 layers/
    core/                # Shared components, composables, styles
 server/                    # Backend API and utilities
    api/                 # Endpoint handlers (grouped by resource)
    utils/               # Database, initialization, helpers
 db/                        # SQLite database file
 docs/                      # Project documentation
 public/                    # Static assets
 scripts/                   # Utility scripts (db seeding, etc.)
 package.json, tsconfig.json, nuxt.config.ts
```

Key Directories

- **apps/**: Three independent Nuxt applications, each with their own pages, layouts, components, middleware, and composables.
- **layers/core/**: Shared UI components, authentication composables, theme variables, and global styles reused across all three apps.
- **server/api/**: Vertical slices of endpoints organized by feature (auth, courses, assignments, staff, availability, feedback).
- **db/**: SQLite database file and schema migrations managed via `server/utils/init.ts`.

Technology Stack

Development Stack

The UMS project uses a modern, type-safe stack optimized for rapid iteration and quality assurance.

Frontend

- **Nuxt 3**: Meta-framework for Vue 3 enabling SSR, file-based routing, and auto-imports.
- **Vue 3**: Component-based UI with Composition API for reactive state management.
- **TypeScript**: Type safety across all frontend code and build steps.
- **TailwindCSS**: Utility-first CSS for rapid UI development and consistent theming.
- **CSS Variables**: Custom theme system (agileblue, agilecyan, agilegreen, etc.) for app-specific branding.

Backend

- **Node.js**: Runtime for server-side endpoint handlers.
- **H3**: Lightweight HTTP server for handling requests (built into Nuxt server).
- **Jose**: JWT library for secure token generation and verification (auth).
- **SQLite**: Embedded database for development and deployment simplicity.
- **TypeScript**: End-to-end type safety from API layer to database.

Development Tools

- **Bun**: Fast package manager and runtime (primary); Node.js/npm as fallback.
- **ESLint**: Code linting and consistency enforcement.
- **Nuxt Typecheck**: Integrated type checking for Nuxt components and modules.
- **Git**: Version control with GitHub for collaboration and CI/CD readiness.

Architecture Patterns

- **API-First**: Backend exposes REST endpoints with JWT authentication; frontend consumes via `$fetch`.
- **Composable-Based Logic**: Shared business logic (auth, navigation, theme) extracted into Vue composables.
- **Component Reusability**: Core components in `layers/core/components/` auto-imported and reused across apps.

- **Middleware Protection**: Auth middleware on protected routes; JWT validation on backend endpoints.

Sprint 2 Summary

Sprint details

Sprint window: 04/May/26 12:00 PM — 18/May/26 11:30 PM (Sprint 2).

PO: Loay Khaled. Scrum Master: Youssef Tarek. Dev Team: Youssef Haddad, Marwan Ahmed, Ziyad Elnemr, Ahmed Sadek.

Sprint Planning — key outcomes

- Reduced scope by removing low-priority features to concentrate on MVP items.
- Top priorities: Security (fix exposed API, strong passwords, API docs), UI/UX (navigation improvements), Usability for instructors, Documentation, Deployment plan.
- Risk items noted: potential quality issues due to deadline changes and misaligned UX expectations.

Daily Scrum highlights (selected)

- 05/05 — Merge conflict on PR referencing FMAP-140; Loay to mediate (evidence: Agile Notes).
- 06/05 — Reduced attendance; meeting canceled.
- 07/05 — Team performed PR reviews and merged outstanding PRs.
- 08/05 — Functional testing of implemented requirements.

Sprint Review

PO feedback: UI/UX improvements accepted; PO liked theme and calendar feature; navigation still noted as needing further simplification.

Sprint Retrospective

Team noted time pressure from schedule change; teamwork improved; action: increase meeting attendance and improve PR coordination.

Meeting Minutes — Formalized

Sprint Planning (04/May/26)

Attendees: PO (Loay), SM (Youssef), Dev Team. Decisions:

- Reduced scope, prioritized security and navigation.
- Estimation approach: Planning Poker (see Estimation section).
- Action items: Fix exposed API endpoints (owner: Marwan, due: 07/May/26).
- Update onboarding docs and API docs (owner: Ziyad, due: 10/May/26).

Daily Scrums (selected entries)

- 05/May/26: Merge conflict on FMAP-140 — action: Loay mediate and produce merge plan.
- 07/May/26: PR review day — all devs to merge and run functional tests.

Sprint Review (18/May/26)

Attendees: PO, SM, Dev Team. Demonstrated: calendar feature, login improvements, assignment deadline view. PO accepted several items; navigation improvement remains.

Sprint Retrospective (18/May/26)

What went well: collaboration under tight deadlines. What to improve: more regular syncs, clearer PR ownership. Action: schedule two mandatory mid-sprint checkpoints.

User Story Splitting

Why We Split User Stories

Large user stories (Epics) cannot be delivered within a single sprint. The team applied recognized splitting patterns to break large stories into independently deliverable, testable increments, each satisfying the INVEST criteria.

INVEST Criteria

Each split story must be: **I**ndependent • **N**egotiable • **V**aluable • **E**stimable • **S**mall • **T**estable

Splitting Techniques Applied

The team applied five core splitting techniques, each demonstrated with examples from the project backlog and sourced directly from the Curriculum, Facilities, and Staff module documentation.

11.2.1 Technique 1 — Split by Role-Based Personas

Split an epic into distinct features based on different user personas with conflicting goals. Each persona gets a focused feature that serves their specific workflow.

Example: Course Catalog Management (Curriculum Module)

Original Epic: *"As a user, I want to manage and access the course catalog."*

1. **Feature A (Admin Persona):** Course Catalog Administration
   - **FMAP-133** — Create a new course MUST
   - **FMAP-137** — View the list of active courses MUST
   - **FMAP-131** — Update existing course details SHOULD
2. **Feature B (Student Persona):** Student Registration Portal
   - **FMAP-132** — Search and filter the course catalog MUST
   - **FMAP-135** — Enroll in a selected course MUST

The epic naturally serves two completely different user personas (admin building curriculum vs. student consuming it), so it is split into two distinct features, each optimized for their workflow.

11.2.2 Technique 2 — Split by Workflow Steps

Decompose a user journey into sequential phases, creating one story per phase so each can be developed and tested independently.

Example 2a: Transcript Management (Facilities Module)

Original Epic: *"As a student, I want to request, track, and receive an official transcript."*

1. **FMAP-196** — Student *requests* an official sealed transcript MUST
2. **FMAP-202** — Admin *processes* and digitally signs the transcript MUST
3. **FMAP-198** — Registrar *updates* student information prior to generation MUST

Each step is testable in isolation and delivers value to a distinct user role.

Example 2b: Performance Tracking (Staff Module)

Original Epic: *"As a professor, I want to submit performance data for annual review."*

1. **FMAP-182** — *Submission Phase*: Add completed course details MUST
2. **FMAP-185** — *Report Phase*: Admin extracts performance report for review WON'T

Workflow split into submission phase (faculty action) and reporting phase (admin action).

11.2.3 Technique 3 — Split by Data Operations (CRUD)

Separate Create, Read, Update, and Delete operations into individual stories, each with distinct complexity and priority.

Example 3a: Course Catalog (Curriculum Module)

Original Epic: *"As an Admin, I want to fully manage the course catalog."*

1. **FMAP-133** — *Create*: Create a new course MUST
2. **FMAP-137** — *Read*: View the list of active courses MUST
3. **FMAP-131** — *Update*: Update existing course details SHOULD
4. **FMAP-130** — *Delete*: Archive a retired course WON'T

Core operations (Create, Read) are prioritized first; complex operations (Update, Delete) follow.

Example 3b: Staff Directory (Staff Module)

Feature: *"Staff directory and role management."*

1. **FMAP-184** — *Read Only*: View centralized staff directory MUST
2. **FMAP-186** — *Read & Write*: Manage course duties and responsibilities MUST

Data Operations applied at varying complexity levels: read-only access is separate from read-write management.

11.2.4 Technique 4 — Split by Business Rules

Deliver core business logic first; add validation and complex rules in subsequent stories to incrementally build sophistication.

Example: Course Enrollment (Curriculum Module)

Original Epic: *"As a student, I want to find and enroll in courses."*

1. **FMAP-132** — *Core Logic*: Search and filter the course catalog MUST
2. **FMAP-135** — *Enrollment*: Enroll in a selected course MUST
3. **FMAP-134** — *Validation*: System validation of course prerequisites SHOULD
4. **FMAP-136** — *Enhancement*: View remaining course capacity/seats COULD

Basic search and enrollment are released first; prerequisite validation and capacity checks are added as subsequent enhancements.

11.2.5 Technique 5 — Split by Simple vs. Complex Implementation

Isolate risky or complex features by splitting them into a simple, internal-only version first, then adding complex integrations in subsequent stories.

Example 5a: Content Delivery (Curriculum Module)

Original Epic: *"As a professor and student, I want to access and manage learning materials online."*

1. **Simple Path (Internal):**
   - Upload and store course materials internally
   - Students access and download materials
2. **Complex Path (External Integration):**
   - Configure and synchronize with external LMS
   - Handle real-time data sync and legacy content migration

Simple, low-risk internal features are delivered first; complex external integrations are added in later sprints.

Example 5b: HR Benefits (Staff Module)

Feature: *"HR benefits and payroll management."*

1. **FMAP-187** — *Simple*: View benefits summary (static page) COULD
2. **FMAP-188** — *Complex*: View payslips with secure access control SHOULD

Simple, static benefits view is quick to deliver; complex payroll with security rules follows.

Splitting Technique Summary

Technique	Best Applied When	Example in Project
Role-Based Personas	Different actors with conflicting goals	Course catalog management
Workflow Steps	Sequential multi-phase user journey	Transcript, Performance tracking
Data Operations (CRUD)	Full data management lifecycle	Course catalog, Staff directory
Business Rules	Core logic before validation rules	Course enrollment
Simple vs. Complex	Risk isolation before complex integration	Content delivery, HR benefits

Estimation Technique

The team used Planning Poker during Sprint Planning:

- Scale: Fibonacci-like (1,2,3,5,8,13).
- Process: PO reads story → discussion → independent votes → reveal → discuss outliers → re-vote until consensus.
- Evidence: `Agile Notes.md` records estimation decisions; merge conflict on FMAP-140 shows a story that required re-estimation during sprint.

Definition of Done (DoD) — Sprint 2 (applies to repository)

Updated DoD (applies to every story/PR merged for Sprint 2):

1. Code compiles and lints: `npx eslint .` — no new lint errors in changed files.
2. Type checks: `npx nuxt typecheck apps/students` (and `apps/staff`, `apps/instructors`) — no type errors in touched modules.
3. Manual testing: stories are manually tested by developers and QA — evidence of testing provided in PR description or manual verification screenshots.
4. API Security: server endpoints validate JWT using `jose` and token checks (see `server/api/*`) — include code snippet and tests for auth where applicable.
5. DB migrations: any required DB schema changes included and seeded via `server/utils/init.ts`.
6. Documentation: update `docs/` or `README.md` with usage or configuration changes.
7. PR review: at least one review approval, CI green (if CI configured), and documented acceptance criteria met.
8. Evidence: PR contains screenshots or links to manual verification and references corresponding FMAP ID(s).

Reference commands:

```
npx eslint .
npx nuxt typecheck apps/students
bun run dev:students
bun run dev:staff
bun run dev:instructors
```

Verification & Evidence

Where to find evidence:

- GitHub repo: `https://github.com/Ziyad-Elnemr/luemy` — check PRs/commits referencing FMAP IDs.
- Server endpoints: `server/api/` — JWT verification code pattern using `jose`.
- DB schema: `server/utils/init.ts`.
- Frontend components implementing features: `layers/core/components/`, `apps/*/components/` and `apps/*/pages/`.
- Jira board (reference): `https://ziyadelnemr90.atlassian.net/jira/software/c/projects/FMAP/boards/35`

Appendix: reproduction commands (run locally)

```
# Install dependencies (repo uses bun in dev; node alternative may be used)
bun install
# Prepare Nuxt (if required)
npx nuxt prepare
# Run lint
npx eslint .
# Typecheck
npx nuxt typecheck apps/students
# Run dev for students app
bun run dev:students
```

Thank You